

NeuraTrade — Confrontações Teóricas

Defesa das decisões de modelagem: dados, arquitetura, validação e estratégia de múltiplos modelos

Projeto NeuraTrade — Redes Neurais (2026.1)

23 de junho de 2026

Resumo

Este documento responde, de forma direta e crítica, às principais objeções teóricas que o desenho do NeuraTrade pode receber. Cada seção abre com a **Confrontação** (a pergunta difícil), apresenta a resposta fundamentada e, quando cabe, registra explicitamente o **Trade-off** aceito e a **Limitação assumida**. O princípio é o mesmo do projeto: preferir a honestidade metodológica à aparência de completude. Detalhes de implementação remetem aos *Architecture Decision Records* (ADRs) em `docs/adr/`; a fundamentação didática dos conceitos está no *Guia Teórico* (`teoria/`).

Sumário

1	Como ler este documento	2
2	Base de dados extremamente desbalanceada	2
2.1	A resposta: o problema foi reformulado, não “balanceado”	2
2.2	Por que não as técnicas usuais de balanceamento	2
2.3	Onde o desbalanceamento reaparece — e como o tratamos	3
3	Escolha e desenho da arquitetura	3
3.1	Por que um LSTM-Autoencoder	3
3.2	Por que os nós de entrada (a representação de entrada)	3
3.3	Profundidade da rede e neurônios por camada	4
4	Metodologia de validação	5
4.1	Três níveis de separação	5
4.2	Validação cruzada para seleção de modelos	6
4.3	Validação substantiva: além da perda	7
5	Múltiplas redes vs. uma rede única	7
5.1	A razão de fundo: “normalidade” é específica do ativo	7
5.2	O objetivo do projeto exige a separação	7
5.3	Os contra-argumentos, honestamente	7
6	Síntese e limitações assumidas	8
6.1	O que ficou em aberto (e por quê)	8

1 Como ler este documento

As perguntas aqui enfrentadas são do tipo que uma banca faz para testar se o projeto entende as *consequências* de suas escolhas — não apenas se seguiu uma receita. Por isso cada decisão é defendida junto com o que ela *custa* e o que ela *deixa de fora*.

Recapitulando o objeto em uma linha: o NeuraTrade detecta anomalias de forma **não supervisionada** em quatro ações da B3 (PETR4, VALE3, AMER3, ITUB4), treinando um **LSTM-Autoencoder por ativo** sobre log-retornos de 2010–2019 (“normalidade”) e sinalizando, em 2020–2024, as janelas cujo **erro de reconstrução** ultrapassa um limiar.

Mapa das confrontações:

1. Base de dados extremamente desbalanceada (Seção 2).
2. Escolha e desenho da arquitetura (Seção 3): por que LSTM-Autoencoder, por que os nós de entrada, profundidade e largura.
3. Metodologia de validação (Seção 4), incluindo o papel da validação cruzada na seleção de modelos.
4. Por que múltiplas redes em vez de uma rede única (Seção 5).

2 Base de dados extremamente desbalanceada

Confrontação. Anomalias são, por definição, raríssimas: talvez 1% dos dias, ou menos. Em classificação supervisionada, uma classe tão minoritária arruína o treino — o modelo aprende a dizer sempre “normal” e acerta 99%. Como o projeto lida com um desbalanceamento tão extremo?

2.1 A resposta: o problema foi reformulado, não “balanceado”

A premissa da pergunta é de *classificação supervisionada*. O NeuraTrade **não** é supervisionado e **não** classifica em duas classes. A estratégia evita o desbalanceamento na origem, reformulando a tarefa como **aprendizado de uma classe** (*one-class learning*) via reconstrução:

- Treina-se **somente com dados “normais”** (2010–2019). A classe minoritária (anomalias) *nem entra* no treino — logo não existe proporção entre classes para desbalancear.
- O modelo aprende a **reconstruir** o normal. A anomalia é detectada pelo *fracasso* de reconstrução (erro alto), não por um rótulo de classe. É a mesma lógica de detecção de fraude/falha por *novelty detection*.

Intuição. Em vez de mostrar ao modelo milhões de dias normais e um punhado de anômalos (e implorar para ele não ignorar o punhado), mostramos *só* o normal e deixamos a anomalia ser “tudo aquilo que ele não consegue reproduzir”. O desequilíbrio de contagem deixa de ser uma variável do treino.

2.2 Por que não as técnicas usuais de balanceamento

As ferramentas clássicas contra desbalanceamento seriam contraproducentes aqui:

- **Oversampling / SMOTE:** sintetizaria “anomalias” interpolando exemplos — mas não temos exemplos rotulados de anomalia, e interpolar em série temporal quebraria a estrutura sequencial.
- **Undersampling do normal:** jogaria fora justamente os dados que *definem* a normalidade que queremos aprender.
- **Pesos de classe:** pressupõem duas classes no treino, que não existem na formulação *one-class*.

2.3 Onde o desbalanceamento reaparece — e como o tratamos

Ele não some de todo: ressurge na **avaliação**. Ao injetar 50 anomalias sintéticas numa série de ≈ 1200 janelas de teste, o *ground-truth* é fortemente desbalanceado ($\approx 4\%$ positivos). Isso tem duas implicações que o projeto trata explicitamente:

1. **Acurácia é inútil** num cenário assim (dizer “tudo normal” dá $\approx 96\%$ de acurácia e zero utilidade). Por isso reportamos **Precision, Recall e F1**, não acurácia.
2. **Teto de precisão pela taxa-base**. O limiar p95 marca $\approx 5\%$ das janelas por construção; contra $\approx 4\%$ de positivos reais, há um piso de falsos positivos que limita a *precision* máxima. Esse número é reportado *junto* das métricas, para não inflar a leitura.

Trade-off. O paradigma *one-class* resolve o desbalanceamento, mas paga um preço: como nunca vemos anomalias rotuladas, não há como o modelo aprender a *discriminar* tipos de anomalia, nem otimizar diretamente uma fronteira de decisão. Ele só sabe “isto não parece normal”. Para um problema sem rótulos confiáveis (o nosso caso), é a troca certa; para um cenário com rótulos abundantes, um classificador supervisionado com ponderação de classe poderia superar.

3 Escolha e desenho da arquitetura

3.1 Por que um LSTM-Autoencoder

Confrontação. Há dezenas de detectores de anomalia. Por que justamente um autoencoder recorrente, e não um método estatístico clássico, um classificador, ou outra arquitetura de rede?

A escolha resulta do cruzamento de três exigências do problema, que eliminam as alternativas uma a uma:

1. **Não supervisionado** (não há rótulos). Isso descarta classificadores supervisionados (*random forest*, redes de classificação) e métodos que exigem exemplos de anomalia.
2. **Sequencial / temporal** (a ordem dos dias carrega informação; há dependência e regimes de volatilidade que se estendem por semanas). Isso enfraquece métodos que tratam cada dia como ponto independente (*Isolation Forest*, *One-Class SVM* sobre *features* estáticas, *z-score* pontual).
3. **Padrão complexo e não linear** de “normalidade”. Isso desfavorece modelos lineares puros (ARIMA, limiar fixo sobre o retorno).

O **autoencoder** atende (1): aprende a normalidade por reconstrução, sem rótulos. A parte **LSTM** atende (2) e (3): processa a janela como sequência, com memória de longo prazo e não-linearidade. A combinação — comprimir uma janela temporal e medir o erro ao reconstruí-la — é o casamento natural das três exigências.

Intuição. A LSTM dá “memória” e a estrutura de autoencoder dá “aprender só o normal”. Tirar qualquer uma das duas quebra um requisito: sem LSTM, ignoramos o tempo; sem autoencoder, precisaríamos de rótulos.

3.2 Por que os nós de entrada (a representação de entrada)

Confrontação. A entrada da rede é um tensor de forma (30, 1): 30 passos no tempo, 1 variável por passo. Por que uma única variável (e não OHLCV completo)? Por que 30 passos?

Tabela 1: Por que as alternativas foram preteridas.

Alternativa	Motivo de não adoção
z -score / média móvel	Linear e pontual; serve de <i>baseline</i> (e foi implementado como tal), mas não capta padrão sequencial.
ARIMA / GARCH	Fortes em séries financeiras, mas assumem forma paramétrica; o objetivo do projeto é justamente uma abordagem <i>neural</i> de representação aprendida.
Isolation Forest / OC-SVM	Não supervisionados, porém sobre vetores de <i>features</i> sem ordem temporal nativa.
AE com camadas densas (MLP)	Perde a estrutura sequencial: trataria os 30 passos como 30 <i>features</i> independentes.
AE convolucional (1D-CNN)	Capta padrão local, mas a dependência de longo alcance é melhor servida pela memória explícita da LSTM.
VAE / GAN	Mais expressivos, porém mais difíceis de treinar e fora do escopo da proposta; registrados como trabalho futuro.

Uma variável: o log-retorno do fechamento. A entrada é o **log-retorno diário** $r_t = \ln(P_t/P_{t-1})$, e só ele. Justificativas:

- **Estacionariedade.** O preço bruto tem tendência (cresce ao longo dos anos); o log-retorno remove essa tendência, dando à rede um alvo de “normalidade” estável no tempo. Treinar sobre preço faria o normal de 2010 (preços baixos) parecer anômalo em 2019.
- **Comparabilidade entre ativos** e imunidade a *splits* (o log transforma fatores multiplicativos em deslocamentos aditivos).
- **Parcimônia.** Com ≈ 2450 janelas de treino por ativo, uma entrada univariada mantém o número de parâmetros compatível com o volume de dados (Seção 3.3), reduzindo risco de *overfitting*.

Por que não OHLCV (multivariado). Volume e preços máximo/mínimo carregam informação real, mas: (i) multiplicariam os parâmetros sem aumentar os dados; (ii) exigiriam normalização conjunta cuidadosa; (iii) tornariam a detecção menos interpretável (erro misturando escalas diferentes). A decisão é *deliberadamente univariada* — e sua principal consequência (anomalias puramente de volume são invisíveis) é assumida e registrada como trabalho futuro (autoencoder multivariado).

Por que 30 passos. A janela $L = 30$ (≈ 6 semanas úteis) é a granularidade da “frase” que a LSTM lê. É o valor consagrado nas implementações de referência de detecção com LSTM (Li, 2020; Valkov) e equilibra: janela curta demais não captura regime; longa demais dilui eventos pontuais e reduz o número de janelas de treino. A forma final $(n, 30, 1)$ é exatamente o que a camada LSTM espera.

Trade-off. Univariado + janela de 30: maximiza estabilidade e parcimônia, ao custo de cegueira a sinais fora do retorno do fechamento (volume, *intraday*) e a anomalias mais curtas que a resolução da janela.

3.3 Profundidade da rede e neurônios por camada

Confrontação. Por que uma rede tão rasa? Por que 64 unidades nas LSTMs e um gargalo de 16? Como esses números foram escolhidos — e não chutados?

A rede tem **38.737** parâmetros, distribuídos assim:

Tabela 2: Arquitetura camada a camada (entrada (30, 1), um modelo por ativo).

Camada	Saída	Params	Papel
Input (janela)	(30, 1)	0	janela de log-retornos
LSTM (encoder, 64)	(64)	16.896	comprime a sequência em um vetor
Dropout (0,2)	(64)	0	regularização
Dense (gargalo, 16, ReLU)	(16)	1.040	<i>bottleneck</i> : força a compressão
RepeatVector(30)	(30, 16)	0	replica o latente para o decoder
LSTM (decoder, 64, seq)	(30, 64)	20.736	reconstrói passo a passo
Dropout (0,2)	(30, 64)	0	regularização
TimeDistributed Dense(1)	(30, 1)	65	projeta de volta ao espaço do retorno
Total		38.737	

Profundidade (rasa) é proporcional aos dados. São ≈ 2450 janelas de treino por ativo. Uma regra prática saudável é manter os parâmetros na mesma ordem de grandeza (ou abaixo) do produto amostras $\times$ sinal, sob risco de *overfitting*. Com $\approx 38,7$ mil parâmetros para ≈ 2450 janelas de 30 passos, já estamos num regime em que a regularização (*dropout*, *early stopping*) importa — aprofundar a rede pioraria isso sem ganho, dado que o sinal (uma série univariada) é simples. Uma camada LSTM em cada ponta é suficiente para a expressividade necessária.

64 unidades: *default* de literatura. `lstm_units=64` e `dropout=0,2` reproduzem a implementação de referência (Valkov), padrão consolidado em AE-LSTM para séries financeiras — proveniência “*default* de literatura”, não arbitrária.

Gargalo de 16: decisão de projeto *verificada*. A literatura não padroniza a dimensão latente. Fixou-se 16 (compressão $\approx 4:1$ sobre as 64 unidades) e, crucialmente, **mediu-se a sensibilidade**: treinando PETR4 com `latent_dim` $\in \{8, 16, 32\}$, a perda de validação foi praticamente idêntica (0,00342/0,00343/0,00342). Ou seja, o gargalo *não é restrição ativa* nessa faixa — até 8 bastaria, e 16 foi mantido com folga. Isso responde diretamente à objeção do “chute”: o valor foi confirmado por experimento, não por fé.

Limitação assumida. O estudo de `latent_dim` foi conduzido em *um* ativo (PETR4) e com um único *seed*. A planura observada é forte evidência, mas uma varredura nos quatro ativos e com múltiplos *seeds* seria mais robusta. Igualmente, `lstm_units` e `dropout` foram herdados da literatura, sem varredura própria — assumido como escopo.

4 Metodologia de validação

Confrontação. Como o projeto valida os modelos? Há um conjunto de teste honesto? A separação respeita a natureza temporal dos dados, ou há vazamento?

4.1 Três níveis de separação

A validação opera em três níveis, deliberadamente distintos:

1. **Treino** (2010–2019): aprende a normalidade.
2. **Validação** (bloco final do treino, $\approx$ 2018–2019, via `validation_split=0,1` com `shuffle=False`): governa o *early stopping* e a calibração de hiperparâmetros. **Não** é o teste.
3. **Teste** (2020–2024): nunca visto no treino; é onde as anomalias são detectadas e as métricas reportadas.

O detalhe crítico: `shuffle=False`. O `validation_split` do Keras separa a *última fração* do array *antes* de embaralhar. Com `shuffle=True`, cada época re-embaralharia o treino e — combinada com janelas sobrepostas — misturaria amostras temporalmente adjacentes entre treino e validação, vazando padrão futuro. Com `shuffle=False`, treino e validação ficam em blocos cronológicos contíguos. Esta é a defesa central contra vazamento, e está registrada como decisão crítica (ADR-0004).

Limitação assumida. Há um vazamento *residual* conhecido: janelas na fronteira treino/validação compartilham alguns dias, por causa da sobreposição de janelas. O efeito é pequeno e foi assumido e documentado, não ignorado.

4.2 Validação cruzada para seleção de modelos

Confrontação. Para escolher o modelo ótimo entre opções pré-determinadas, usou-se *validação cruzada*? Onde está o *k-fold*?

Aqui é preciso ser direto: **não usamos *k-fold* clássico — e isso é intencional, não um esquecimento.**

Por que *k-fold* padrão é inválido aqui. A validação cruzada *k-fold* embaralha e particiona os dados assumindo que as amostras são *independentes e identicamente distribuídas* (i.i.d.). Séries temporais violam isso: as observações são dependentes e ordenadas. Um *fold* de treino contendo dias *posteriores* aos do *fold* de teste constitui vazamento de futuro — exatamente o pecado da Seção 4. Em detecção de anomalias isso é ainda pior, pois infla artificialmente a aparência de desempenho.

O que de fato fizemos: seleção por *grid* sobre validação temporal. A “comparação de múltiplos modelos para escolher o ótimo” aconteceu, mas pela via apropriada a séries temporais — avaliar candidatos numa validação que respeita o tempo:

- **Dimensão latente** $\in \{8, 16, 32\}$: comparadas pela `val_loss` no bloco de validação cronológico; resultado plano, 16 confirmado (Seção 3.3).
- **Esquema de limiar** (estático p95 vs. dinâmico causal) e **tamanho da janela dinâmica** $\in \{126, 252, 504\}$: comparados pela taxa de marcação e pelo comportamento sob mudança de regime no teste; 252 confirmado.
- **Modelo profundo vs. *baseline***: o LSTM-AE é confrontado com um *baseline* de *z-score* em janela móvel, para evidenciar ganho sobre a heurística trivial.

A forma correta de CV temporal (e nosso caminho de rigor). A validação cruzada que *seria* válida aqui é a de série temporal — *walk-forward* / janela expansível, em que cada *fold* treina no passado e valida no futuro imediato, nunca o contrário:

```
1 from sklearn.model_selection import TimeSeriesSplit
2 tscv = TimeSeriesSplit(n_splits=5)
3 for treino_idx, val_idx in tscv.split(X):
4     # treino_idx sempre ANTES de val_idx -> sem vazamento de futuro
5     modelo = build_lstm_autoencoder()
6     modelo.fit(X[treino_idx], X[treino_idx], shuffle=False, ...)
```

```
7 avalida(modelo, X[val_idx])
```

Listing 1: Validação cruzada temporal (walk-forward) com scikit-learn.

Limitação assumida. O projeto usou um **único** *holdout* temporal de validação (não uma *k-fold* temporal com vários cortes). Para a seleção de hiperparâmetros conduzida — de baixa dimensionalidade e com resultados planos/robustos — isso é suficiente e defensável. Porém, um `TimeSeriesSplit` (*walk-forward*) daria estimativas com *variância* (média $\pm$ desvio entre cortes) e uma seleção de modelo estatisticamente mais sólida. Está registrado como o próximo passo de rigor — e o ADR-0004 já o aponta como alternativa considerada.

4.3 Validação substantiva: além da perda

Vale frisar que a `val_loss` não é o único juiz. Como não há rótulos de anomalia reais, a validação do *detector* (não só do reconstrutor) se dá por duas vias complementares (detalhadas no relatório principal):

- **Quantitativa:** injeção sintética de anomalias em posições conhecidas $\rightarrow$ Precision/Recall/F1.
- **Narrativa:** as anomalias detectadas coincidem com eventos reais (COVID, fraude da Americanas, Brumadinho)?

Essa dupla checagem é a resposta honesta para “como validar um modelo sem gabarito”.

5 Múltiplas redes vs. uma rede única

Confrontação. Treinar quatro modelos separados (um por ativo) desperdiça dados e esforço. Por que não uma única rede, treinada com os quatro ativos juntos, que generalizaria melhor?

5.1 A razão de fundo: “normalidade” é específica do ativo

O detector funciona aprendendo *o que é normal*. Mas a normalidade de cada ativo é distinta: a volatilidade típica, a autocorrelação e o regime de risco de PETR4 (energia, sensível a petróleo e política) não são os de ITUB4 (banco, mais estável) nem os de AMER3 (varejo, que entrou em colapso). Uma rede única aprenderia uma normalidade *média* — e a média de comportamentos heterogêneos não descreve bem nenhum deles.

Intuição. Um médico que atende só cardiologistas aprende com precisão o “normal” daquele grupo. Um clínico que mistura recém-nascidos, atletas e idosos num só padrão de referência classificaria mal todos: o batimento normal de um bebê alarmaria como anômalo num idoso. Cada ativo é uma “população” com sua fisiologia.

5.2 O objetivo do projeto exige a separação

A proposta do NeuraTrade é **comparar a generalização entre setores** (energia, mineração, varejo, financeiro). Essa comparação *só faz sentido* com um modelo por ativo: é o que permite afirmar “o detector de AMER3 cobre 9 de 10 dos seus eventos” ou “um choque sistêmico (COVID) acende em todos os quatro”. Uma rede única dissolveria a unidade de análise (o ativo/setor) que é o próprio objeto de estudo.

5.3 Os contra-argumentos, honestamente

A rede única tem vantagens reais, que reconhecemos:

Tabela 3: Modelo por ativo (adotado) vs. rede única (alternativa).

	Um modelo por ativo (adotado)	Rede única (todos os ativos)
Especificidade	Alta: capta o regime de cada ativo	Baixa: aprende uma média
Dados por modelo	≈ 2450 janelas	≈ 9800 janelas ($4\times$)
Comparação setorial	Direta e interpretável	Indireta / perdida
Risco de <i>overfitting</i>	Maior (menos dados)	Menor (mais dados)
Custo / manutenção	4 modelos para treinar e versionar	1 modelo
Transferência entre ativos	Nenhuma	Implícita (pode ajudar ativos com pouco histórico)

Por que, no saldo, a separação vence aqui. O volume de dados por ativo (≈ 2450 janelas) é *suficiente* para a rede pequena adotada (Seção 3.3), então o principal trunfo da rede única (mais dados) não é decisivo. Já a especificidade da normalidade e a exigência de comparação setorial são centrais ao problema. O custo de manter quatro modelos pequenos é baixo e a configuração é centralizada (um `config.yaml`, o mesmo código, só o ativo muda).

Trade-off. Modelo por ativo troca “mais dados e um artefato único” por “especificidade e comparabilidade setorial”. Para um problema cujo *objetivo declarado* é comparar setores, a troca é favorável. Num cenário com centenas de ativos, ou com ativos de histórico curto, uma **arquitetura intermediária** — um modelo global com *embedding* de ativo, ou *fine-tuning* por ativo sobre uma base compartilhada — seria o próximo passo natural.

6 Síntese e limitações assumidas

As confrontações compartilham um fio condutor: cada decisão do NeuraTrade otimiza para o *problema real* — detecção não supervisionada, em séries temporais, com normalidade específica por ativo — e não para um *checklist* genérico de boas práticas. Resumindo as defesas:

- **Desbalanceamento:** dissolvido pela reformulação *one-class* (treina só no normal); re-appeare na avaliação e é tratado com P/R/F1 e reporte da taxa-base, nunca acurácia.
- **Arquitetura (LSTM-AE):** única combinação que satisfaz simultaneamente “sem rótulos”, “sequencial” e “não linear”. Entrada univariada e janela de 30 por estacionariedade e parcimônia. Rede rasa e gargalo 16 dimensionados ao volume de dados, com a dimensão latente *verificada* por sensibilidade.
- **Validação:** três níveis com separação temporal estrita (`shuffle=False`); seleção de modelos por *grid* sobre validação temporal; *k*-fold clássico evitado por ser inválido em séries temporais.
- **Múltiplas redes:** a normalidade é específica do ativo e a comparação setorial é o objetivo — ambos exigem um modelo por ativo.

6.1 O que ficou em aberto (e por quê)

Listamos junto, para não dispersar, as limitações assumidas ao longo do texto:

1. **Validação cruzada temporal completa** (`TimeSeriesSplit` / *walk-forward*) em vez do *holdout* único, para estimativas com variância (Seção 4.2).

2. **Sensibilidade de hiperparâmetros mais ampla:** `latent_dim` nos quatro ativos e múltiplos *seeds*; varredura de `lstm_units` e `dropout` (Seção 3.3).
3. **Entrada multivariada (OHLCV):** habilitaria detectar anomalias de volume, hoje invisíveis (Seção 3.2).
4. **Erro por máximo na janela** (em vez da média), para combater a diluição de choques pontuais e elevar o *recall*.
5. **Arquitetura intermediária** entre “um por ativo” e “rede única” (modelo global com *embedding* de ativo), relevante ao escalar para muitos ativos (Seção 5).

Nenhum desses itens invalida os resultados atuais; todos são caminhos de fortalecimento. O compromisso do projeto é declarar essas fronteiras explicitamente — a alternativa (silenciá-las) seria a única falha metodológica de fato grave.